

Chapter 6. Troubleshooting Containers





Outline

- Read container logs and troubleshoot common container problems.
- Configure a remote debugger during application development.



Troubleshoot Container Startup

- A container might **not be able to start** successfully for different reasons, for example, due to **missing configuration** or a **file access issue**.
- Depending on the containerized application, the process executed by the container can either **exit with an error status** or **keep running in an inconsistent state**. You can list running and stopped containers by using the **podman ps -a** command.
- If the container is in the **Exited status** then the problem might be in the **start-up process**. To access this information you can use the **podman logs** command.
- **Podman logs <container-name/ID>** is used to see logs of container. You can also use **-f** (**--follow**) to follow log output.



Troubleshoot Container Networking

- **Common container networking issues include the following situations:**
 - 1) Incorrect port mapping.
 - 2) No network access between containers.
 - 3) Host name resolution problems (DNS).



1) Port Mapping Issues

- Developers must distinguish between the following port configuration:
 - 1) Ports **inside the container** that the application opens.
 - 2) Podman port mapping configuration, which **opens a port on the host** and maps it to the **application port inside the container**.
- If the port you assign to the container in the port mapping configuration **does not match** the application port, then the containerized application **fails to communicate with the host**.
- `podman run -d --name httpd -p 8089:8080 registry.access.redhat.com/ubi8/httpd-24:latest`
- You can use the **podman port CONTAINER** command to list the current container port mapping.
 - `podman port CONTAINER`
 - `8000 (container port)/tcp -> 0.0.0.0:8080 (host port)`
- you cannot use Podman commands to **verify the port that the application uses**.
- `podman inspect CONTAINER --format '{{.State.Pid}}'` is to **get the container PID**.



1) Port Mapping Issues

- To verify the application ports in use, list the open network ports in the running container.
- Use Linux commands such as the socket statistics (ss) command to **list open ports and show network connections**. A **socket** is the **combination of a port and an IP** address.
- The ss command lists the open sockets in a system. You can provide the ss command with options to filter and produce the desired output:
 - -p: display the process using the socket
 - -a: display listening and established connections
 - -n: display IP addresses
 - -t: display TCP sockets
- **podman exec -it CONTAINER ss -pant** is used to verify that your port mapping matches the application ports in use



Demo for using ss command

- Create a container named **ss-demo** in the background using **ubuntu:22.04** image.
 - **podman run -d -it --name ss-demo ubuntu:22.04**
- Go inside the container.
 - **podman exec -it ss-demo /bin/bash**
- Install ss command. SS command comes with **iproute2** package.
 - **apt update**
 - **apt install iproute2 -y**
 - **ss -pant** , local address is the container ip (hostname -i)

State	Recv-Q	Send-Q	Local Address:Port	Peer
Address:Port	Process			
TIME-WAIT	0	0	10.0.2.100:33872	91.189.91.82:80
TIME-WAIT	0	0	10.0.2.100:46604	185.125.190.81:80



2) Container Network Connectivity Issues

- Developers commonly use Podman networks to **isolate container network traffic** from the host.
- If a container does **not attach to a Podman network**, then it **cannot communicate** with other containers on that network.
- **podman inspect CONTAINER --format='{{.NetworkSettings.Networks}}'** is used to verify that every container is using a specific network.



2) Container Network Connectivity Issues

- cont1 and cont2 are using host NAT network (10.0.2.100) → cannot see each other by container IP
- cont3 and cont4 are on Podman bridge network (cni-podman0) → can see each other using IP

```
marwa@marwa:~$ podman ps
CONTAINER ID   IMAGE                                     COMMAND                  CREATED        STATUS        PORTS                               NAMES
3ac44a6680be   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-httpd...  8 minutes ago Up 8 minutes ago 0.0.0.0:8089->8080/tcp cont1
b62c57371c5f   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-httpd...  8 minutes ago Up 8 minutes ago 0.0.0.0:8088->8080/tcp cont2
2e59969bf595   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-httpd...  3 minutes ago Up 3 minutes ago 0.0.0.0:8081->8080/tcp cont3
e28390858f9e   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-httpd...  2 minutes ago Up 2 minutes ago 0.0.0.0:8082->8080/tcp cont4

marwa@marwa:~$ podman exec -it 3ac44a6680be /bin/bash
bash-4.4$ hostname -i
10.0.2.100
bash-4.4$ exit
exit
marwa@marwa:~$ podman exec -it cont2 /bin/bash
bash-4.4$ hostname -i
10.0.2.100
bash-4.4$ exit
exit
marwa@marwa:~$ podman exec -it cont3 /bin/bash
bash-4.4$ hostname -i
10.89.0.2
bash-4.4$ exit
exit
marwa@marwa:~$ podman exec -it cont4 /bin/bash
bash-4.4$ hostname -i
10.89.0.3
bash-4.4$ exit
exit
```



3) Name Resolution Issues

- Podman networks provide connectivity for container-to-container network traffic by **using IP addresses**.
- However, because **IP addresses might change**, developers **use host names to address these containers in a predictable way**. The Domain Name System (DNS) service **translates container names to IP addresses** for network communication.
- If the network in use **does not have DNS enabled**, **containers cannot resolve the container hostnames and cannot communicate**.
- **podman network inspect <network-name/id>** is used to ensure that DNS is enabled for a Podman network use the podman network inspect command.
 - "dns_enabled": true,



3) Name Resolution Issues

- DNS is enabled in mynet network, so you can **use host names to address this container.**

```
marwa@marwa:~$ podman ps
CONTAINER ID   IMAGE                                     COMMAND                                     CREATED        STATUS        PORTS        NAMES
3ac44a6680be   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-http... 20 minutes ago Up 20 minutes ago 0.0.0.0:8089->8080/tcp cont1
b62c57371c5f   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-http... 19 minutes ago Up 19 minutes ago 0.0.0.0:8088->8080/tcp cont2
2e59969bf595   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-http... 14 minutes ago Up 14 minutes ago 0.0.0.0:8081->8080/tcp cont3
e28390858f9e   registry.access.redhat.com/ubi8/httpd-24:latest /usr/bin/run-http... 14 minutes ago Up 14 minutes ago 0.0.0.0:8082->8080/tcp cont4

marwa@marwa:~$ podman network ls
NETWORK ID     NAME      VERSION    PLUGINS
2f259bab93aa   podman    0.4.0      bridge,portmap,firewall,tuning
11c844f95e28   mynet     0.4.0      bridge,portmap,firewall,tuning,dnsname

marwa@marwa:~$ podman exec -it cont3 /bin/bash
bash-4.4$ curl http://cont4:8080
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN" "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en">
  <head>
    <title>Test Page for the HTTP Server on Red Hat Enterprise Linux</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <style type="text/css">
      /*<![CDATA[*]
        body {
          background-color: #fff;
          color: #000;
          font-size: 0.9em;
          font-family: sans-serif,helvetica;
          margin: 0;
          padding: 0;
```



Troubleshoot Bind Mounts

- By default, the Httpd process **has insufficient permissions to access** the `/var/www/html` directory. This can be a file permission issue or an SELinux issue.
- To troubleshoot file permission issues, use the `podman unshare` command to execute the `ls -l` command.
- `podman run -p 8080:8080 --volume /www:/var/www/html registry.access.redhat.com/ubi8/httpd-24:latest => can't ls var/www/html`
- By default, the Httpd process **has insufficient permissions to access** the `/var/www/html` directory. This can be a file permission issue or an SELinux issue.
- **To fix the SELinux configuration, add the `:z` or `:Z` option to the bind mount:**
 - 1) **Lower case z** lets different containers share access to a bind mount.
 - 2) **Upper case Z** provides the container with exclusive access to the bind mount.
- `podman run -p 8080:8080 --volume /www:/var/www/html:Z \`
`registry.access.redhat.com/ubi8/httpd-24:latest`. So, you will have access to the bind mount.

Section 6.3. Remote Debugging Containers





Debugging With VSCodium

- **VSCodium** is an **open source version of the Visual Studio Code text editor**. Both versions of the editor include a built-in debugger that can connect to various runtimes' debug protocols.

Lab: Section 6.5: Troubleshooting Containers

